

K-2SO Animatronic Controller

System Documentation v1.3.0

Complete user handbook for the K-2SO Animatronic Controller. Build, configure, calibrate, record, replay -- everything in one place.

Version 1.3.0 -- 2026-04

Printed-Droid.com

<https://www.printed-droid.com>

Disclaimer and Liability Notice

K-2SO Animatronic Controller is a do-it-yourself project. Building, wiring, configuring, and operating an animatronic system involves work that, if done incorrectly, can cause damage to property or persons -- in particular through servos under load, hot soldering equipment, mains-powered PSUs, and battery packs.

By using this documentation, hardware, and software you acknowledge:

- You are solely responsible for all work on your build.
- You have the required basic knowledge in electronics and Arduino tooling, or you seek qualified help.
- You verify voltage, polarity, fuses, and connectors before every power-on.
- You only use components matching the application (5V/3A PSU minimum, fused supply lines).
- You comply with local electrical regulations.
- You change the default web password before exposing the device on a network.

The author and all contributors accept no liability for damage caused by incorrect wiring, wrong components, improper operation, or insufficient safety measures.

The documentation and software are provided **as is** without warranty of any kind. If in doubt, seek qualified help before working on the system.

Table of Contents

Disclaimer and Liability Notice	2
Table of Contents	3
Introduction -- What is the K-2SO Animatronic Controller?	8
Version Overview	9
Key Features (v1.3.0)	9
Who Should Read This Manual	9
Safety	10
Power Supply	10
Servo Pinch Hazard	10
Mounting in the Droid	10
Network Security	10
Quick Safety Checklist	11
Quick Start	12
Uploading the Sketch	14
Toolchain Options	14
Required Libraries (v1.3.0)	14
Board Settings (Arduino IDE)	14
USB / Driver Issues	15
Common Compile Errors	15
WiFi & Web UI	16
Connecting in STA Mode	16
AP Fallback	16
Reaching the WebUI	16
Authentication	16
WebUI Tour	16
Board & Pin Map	21
GPIO Pin Assignment (v1.3.0)	21

Connector Groups	22
Wiring Notes	23
Power & PSU	24
Current Budget	24
Recommended Topology	24
Battery Builds	24
LED Hardware	25
Eye NeoPixels: 7-LED vs 13-LED	25
Detail LED Strip	25
Status LED	25
Servo Calibration	28
Default Values	28
Calibration Workflow	28
Profiles	29
Eye Animations	30
Animation Modes	30
Color & Brightness	30
Audio (DFPlayer Mini)	31
SD Card Preparation	31
CLI Reference	31
Default Volume & Auto-Mode	31
IR Remote Setup	32
Learning Mode Workflow	32
IR-Related Commands	32
Personality Modes	34
Tuning Mode Behaviour	34
Wake / Sleep Behaviour	34
Sequence Recording (NEW v1.3.0)	35
What is Captured per Frame	35

Capacity	35
Recording Workflow (Serial CLI)	35
Playback Commands	36
Crash-Safe Save (NEW v1.3.0 follow-up)	37
Pause and Resume (Web UI Only)	37
Playlist Chaining (NEW v1.3.0)	39
Building a Playlist	39
Behaviour and Robustness	39
Example Performance	39
IR-to-Sequence Mapping (NEW v1.3.0)	41
Prerequisites	41
Mapping Commands	41
Example	41
Serial CLI Reference	44
Top-Level Commands	44
Hierarchical Commands	44
Common Quick Commands	45
Voice Assistants	46
Trigger Endpoints	46
Setup Example: Google Home / IFTTT	46
Curl / Manual Testing	46
Configuration & Backup	47
What Is Stored	47
Backup via Serial	47
Restore via Serial	47
Profiles vs Backup	48
Migration v1.2.x to v1.3.0	49
What Happens on First Boot After Upgrade	49
Preserved Fields	49
New / Reset Fields	49

Recommended Upgrade Procedure 49

Troubleshooting **51**

Symptom-to-Fix Table 51

Version History **52**

v1.3.0 -- Promotion Candidate (2026-04) 52

v1.2.4 -- Previous Stable (2026-04) 52

v1.2.5 -- Beta (2026-04) 53

v1.2.3 -- Stable (2026-03) 53

v1.1.0 -- Archive (2026-02) 53

Part I

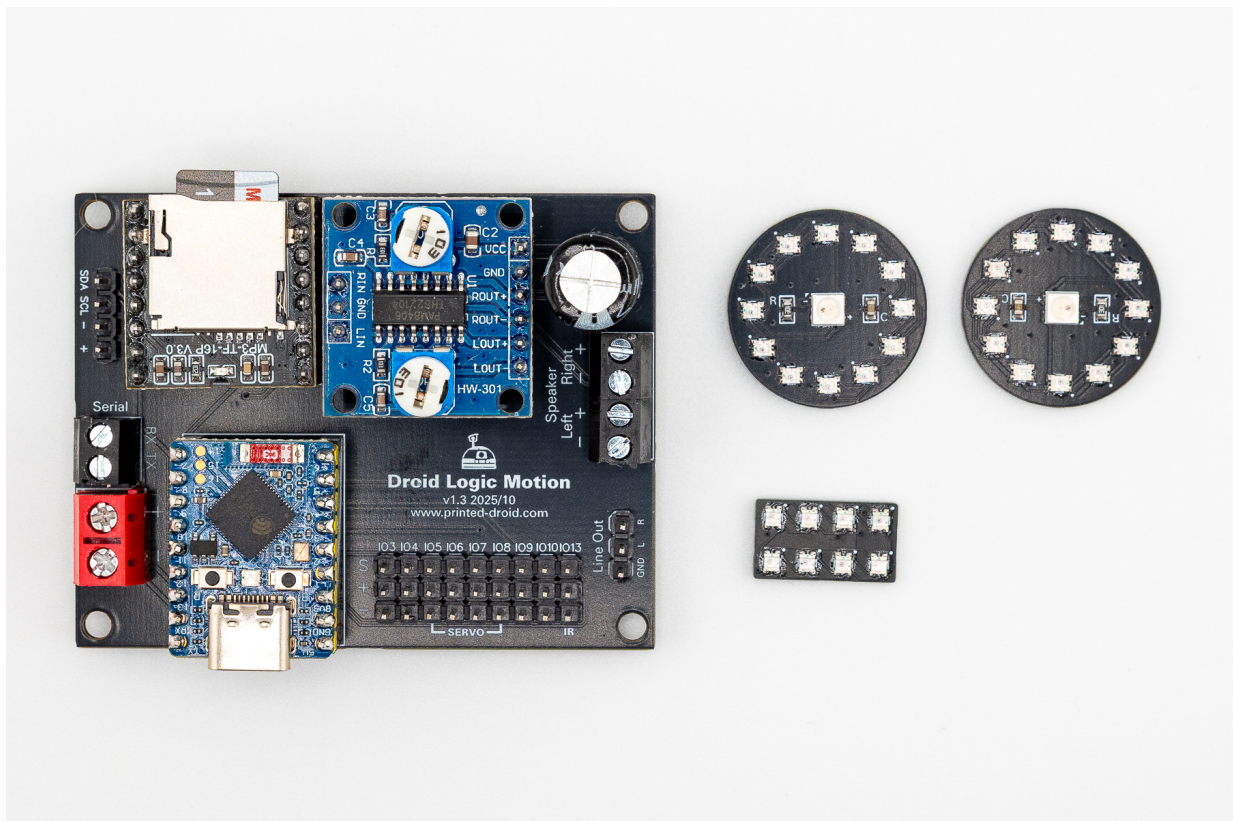
Getting Started

Introduction -- What is the K-2SO Animatronic Controller?

The K-2SO Animatronic Controller is an ESP32-S3 based controller for the K-2SO animatronic from **Droid Division** (sold there as the *Security Droid*). It drives four servos for eye and head motion, two NeoPixel eye rings, a detail-LED strip, a status LED, a DFPlayer Mini for audio, an IR remote receiver, a WiFi web interface, and persistent EEPROM configuration. Since v1.3.0 it also records, replays, and chains custom motion sequences stored in LittleFS.

Where to get the droid: the K-2SO kit and finished figures are sold by Droid Division at <https://www.droiddivision.com/>. Droid Division's own builder community is on Facebook: <https://www.facebook.com/groups/2505708886350784/>.

Controller firmware and support: the source code, releases, and issue tracker for this controller live on GitHub at <https://github.com/PrintedDroid/K-2SO-Animatronic>. Builder questions about the controller and the Droid Logic Motion board are answered in the Printed Droid Facebook group: <https://www.facebook.com/groups/printeddroid/>.



Droid Logic Motion v1.3 controller with eye boards and detail-LED board.

Version Overview

Version	Status	Highlights
v1.1.0	Archive (read-only)	Initial release: basic servo control, simple eye animation, DFPlayer audio, rudimentary serial CLI.
v1.2.4	Previous Stable	Stable patch on top of v1.2.3 with 6 critical and high-severity stability fixes (no-block status LED, restore checksum verify, profile-slot guard, servo-limit min<max, blocking-CLI timeout, detail-LED loop guard). EEPROM compatible with v1.2.3.
v1.3.0 (this manual)	Stable Candidate	Promotes the v1.2.5 beta features to stable: frame-based sequence recording, playlist chaining, IR-button-to-sequence mapping. Adds automatic v1 -> v2 EEPROM migration that preserves all existing data.

Key Features (v1.3.0)

- **4-Servo Motion:** Eye Pan/Tilt + Head Pan/Tilt, calibratable centers and limits, 5 saved profiles.
- **Dual NeoPixel Eyes:** 14 animation modes, runtime-switchable 7-LED or 13-LED hardware.
- **WS2812 Detail Strip:** 1-8 LEDs on GP10, multiple patterns, brightness control.
- **Status LED:** Single WS2812 on GP21 -- shows boot/WiFi/mode/error states at a glance.
- **DFPlayer Audio:** 40+ K-2SO sound clips, volume control, folder/track addressing.
- **IR Remote:** 21-button learning remote, NEC/Sony codecs, button-to-sequence mapping.
- **WiFi WebUI:** STA + AP fallback (K2SO-XXXXXX), HTTP Basic Auth, mDNS k2so.local.
- **Voice Assistants:** 10 trigger endpoints for Google Home / Alexa / Siri integration.
- **Serial CLI:** 115200 baud, ~30 root commands with subcommands, help, status, monitor.
- **Sequence Recording (NEW v1.3.0):** frame-based, up to 200 frames, JSON storage on LittleFS.
- **Playlist Chaining (NEW v1.3.0):** link up to 10 sequences, optional loop.
- **IR-to-Sequence Mapping (NEW v1.3.0):** assign any of the 21 IR buttons to a saved sequence.
- **EEPROM Auto-Migration (NEW v1.3.0):** v1 layout (v1.2.3 / v1.2.4) -> v2 without data loss.

Who Should Read This Manual

This manual targets **builders** who have wired the hardware and want to know how to operate, configure, and extend it. Some Arduino-IDE or PlatformIO familiarity is assumed. The **Developer Reference** (separate PDF) covers EEPROM struct internals, HTTP API endpoints, and library-version constraints for those who want to modify the firmware.

Tip: read the Safety chapter before powering on for the first time. A 5V/3A PSU is the absolute minimum -- weaker supplies cause brownouts during DFPlayer + WiFi boot.

Safety

This chapter lists the practical safety rules for building, wiring, and operating the animatronic. The full liability disclaimer is on page 2.

Power Supply

5V at 3A (15W) is the absolute minimum. Four servos can briefly pull 1.5-2A simultaneously when reversing direction under load. Add the ESP32-S3 (~150 mA peaks during WiFi association), the DFPlayer (250 mA when starting playback), and four NeoPixel strings at maximum brightness, and the 3A budget is realistic, not generous.

Brownout warning: The combination DFPlayer + WiFi boot is power-hungry. With an undersized PSU you will see the ESP32 reset with a "Brownout detector was triggered" message in the serial log. Switch to a stronger PSU, or supply servos and logic separately with a common ground.

Servo Pinch Hazard

Servos are not gentle. During the very first servo test all or after editing limits, keep fingers, cables, and decorative parts clear of the servo horns and any mechanical linkages. Run the test with the head removed from any shoulder mount until you trust the limits.

Mounting in the Droid

The K-2SO from Droid Division is a freestanding, life-size figure. The full head assembly (eyes, NeoPixel boards, mechanics, servos) can still weigh a few hundred grams. Servos behave differently under load than on the workbench: hold-against-gravity torque is reached faster, gear wear shows up earlier. After installing the controller and the LED boards in the head, re-verify the servo limits, re-center, and observe the droid for at least one minute under each personality mode before sending it into a display.

Network Security

The web interface is protected by HTTP Basic Auth with the default credentials `admin / k2so2025`. **Change the password** before connecting the droid to any network outside your workbench. Anybody on the same WiFi can otherwise reach the controls.

Quick Safety Checklist

#	Check	Why
1	5V/3A PSU minimum, fused supply lines	Brownouts kill DFPlayer UART and WiFi association.
2	Verify polarity and pin assignment before first power-on	GPIO short on 5V destroys the ESP32-S3.
3	Servos free-running, no mechanical load for first servo test all	Avoids stripped gears if a limit is wrong.
4	Change k2so2025 default password before networking	Network exposure with the default password is open access.
5	Add a master switch on the 5V rail	You will want to power the droid down quickly more than once.

Quick Start

This chapter takes a freshly flashed v1.3.0 controller from cold to "droid moves and talks" in about 15-20 minutes, assuming the hardware is wired correctly and a microSD with sound files is in the DFPlayer. If something breaks along the way, jump to the Troubleshooting chapter.

Run the first servo test with the head off any mount. Wrong limits can drive servos to mechanical end-stops; the gears will not survive that under load.

1. Install Arduino libraries

Open Arduino IDE, Library Manager. Install: **Adafruit NeoPixel**, **ESP32Servo**, **IRremote**, **DFRobotDFPlayerMini**, **ArduinoJson** (v7). LittleFS ships with the ESP32 core.

2. Add the ESP32 board package

Tools → Board → Boards Manager. Search "esp32", install **esp32 by Espressif Systems** (3.x). Then Tools → Board → ESP32 → **Waveshare ESP32-S3-Zero**. FQBN: esp32:esp32:waveshare_esp32_s3_zero. Pick the correct COM port.

3. Set WiFi credentials

Open `K_2SO_DroidLogicMotion_v1.3.0/config.h`. Edit `WIFI_SSID` and `WIFI_PASSWORD`. You may also leave the placeholders -- the controller starts the K2SO-XXXXXX AP fallback so you can configure WiFi from the browser later.

4. Upload

Upload the sketch. First upload takes a bit longer because the ESP32 toolchain is cold. The Serial Monitor at 115200 baud should show: `=== K-2SO Animatronic Controller v1.3.0 ===`.

5. Watch the boot phases

Status LED transitions: blue blink (boot) → yellow blink (WiFi connecting) → green solid 2s (WiFi up) → mode color (default amber for IDLE / cyan-blue for SCANNING). If yellow blink lasts >30 s, the AP fallback is up.

6. Find the controller on the network

STA: `http://k2so.local` or the IP printed in the serial log. AP fallback: connect your phone to K2SO-XXXXXX (password k2so2025) and visit `http://192.168.4.1`.

7. Log in

Default credentials are admin / k2so2025. **Change the password before exposing the droid to a real network.**

8. First servo test

In Serial Monitor type `servo test all`. The four servos should sweep through their limits one after another. If any servo hits a hard stop or buzzes, stop the droid and edit the limits (see Servo Calibration chapter).

9. First eye test

`led mode pulse --` both eyes should pulse smoothly. `led color 0 0 255` sets blue. `led mode scanner` shows the classic scanner ring.

10. First sound test

`sound play 1 1` plays folder 1, track 1. If silent: SD-card prep is wrong (see Audio chapter). DFPlayer-not-ready boot messages also indicate SD issues.

11. Optional: record your first sequence

Open the WebUI, Sequence Editor, click **Record**, give the sequence a name (e.g. greeting), drive the gamepad sliders, click **Add Frame** with a duration. Repeat. Save. Then **Play**. Full workflow in the Sequence Recording chapter.

Tip: if the boot messages scroll past too fast, type `help` or `status` any time to print the CLI summary or current state.

Uploading the Sketch

This chapter goes deeper than the Quick Start: which IDE, which board package, which libraries and at which versions, common errors with USB / drivers / partition layout, and how the v1.3.0 size budget compares to earlier releases.

Toolchain Options

Two paths are supported: **Arduino IDE 2.x** (easiest for first contact) and **PlatformIO** (better for repeatable builds, multi-version handling). Both produce the same firmware -- the sketch in `K_2SO_DroidLogicMotion_v1.3.0/` compiles in either.

Required Libraries (v1.3.0)

Library	Recommended Version	Notes
Arduino ESP32 core	3.x	Espressif Systems, via Boards Manager. Provides the Waveshare ESP32-S3-Zero target.
Adafruit NeoPixel	latest 1.x	RMT-driver compatible with IRremote. Verified working: 1.15.x.
ESP32Servo	latest 1.x	LEDC channels 0-3 reserved for the four servos.
IRremote	latest 4.x	21-button learning, NEC/Sony codecs.
DFRobotDFPlayerMini	latest 1.x	UART @ 9600 baud on GP11/GP12.
ArduinoJson	7.x	WebUI JSON serialization + sequence file format.
LittleFS	bundled with ESP32 core	Sequence storage on the flash partition.

Library version pinning matters. An older NeoPixel + newer IRremote (or vice versa) can clash on RMT channel allocation. The combination above is verified to coexist; if you change one, retest.

Board Settings (Arduino IDE)

After installing the ESP32 core, choose **ESP32** → **Waveshare ESP32-S3-Zero**. Defaults work; the relevant ones: USB CDC On Boot = Enabled, Flash Size = 4MB, Partition Scheme = **No OTA (2MB APP / 2MB SPIFFS)**, CPU Frequency = 240 MHz. With this partition scheme the v1.3.0 sketch sits at ~60% Flash (1.27 MB / 2 MB) and ~16% RAM. The 2 MB SPIFFS / LittleFS half is used for sequence and playlist storage.

USB / Driver Issues

The ESP32-S3-Zero exposes itself as a USB CDC ACM device. On Windows you may need **CH340** or **CP210x** VCP drivers depending on the USB-to-serial bridge on your board. macOS and Linux usually work out of the box. If the upload says "no port found": hold the BOOT button while plugging USB; the chip enters bootloader mode regardless of the running firmware.

Common Compile Errors

Error	Cause	Fix
ArduinoJson.h not found	Library missing or v6 installed	Library Manager: install ArduinoJson 7.x.
PWM channel taken	NeoPixel allocated LEDC instead of RMT	Use Adafruit NeoPixel 1.15.x or newer (RMT driver).
cannot find esp32:esp32	Board manager URL missing	Add https://espressif.github.io/arduino-esp32/package_esp32_index.json in IDE preferences.
Sketch too big	Wrong partition scheme	Tools → Partition Scheme → "No OTA (2MB APP / 2MB SPIFFS)" . The v1.3.0 sketch declares this in its header as the recommended scheme. Default 4MB with spiffs also works but leaves only ~70 KB free.

Tip: the v1.3.0 sketch declares *No OTA (2MB APP / 2MB SPIFFS)* in its header as the recommended partition scheme. With that, the build sits at ~60% flash (1.27 MB / 2 MB), leaving plenty of headroom for future features without giving up sequence storage on LittleFS.

WiFi & Web UI

The controller offers two WiFi modes: **STA** (joins your home network) and **AP fallback** (starts its own access point named K2SO-XXXXXX when STA fails). The WebUI is the friendly counterpart to the Serial CLI -- everything you can do on serial, you can also do in the browser, plus a gamepad-style servo controller, sequence editor, and IR-map editor.

Connecting in STA Mode

(a) Compile-time: edit `WIFI_SSID` and `WIFI_PASSWORD` in `config.h` before flashing. **(b) Runtime:** after the AP fallback comes up, connect and configure WiFi via the WebUI Settings page, or via the Serial CLI: `wifi set MyNet MyPass` followed by `wifi reconnect`.

AP Fallback

When the controller cannot join the configured WiFi within 30 seconds (or no credentials are set), it starts its own AP. The SSID is generated from the MAC address: K2SO-AABBCC. Default password k2so2025. Connect, then visit `http://192.168.4.1`. From here you can configure WiFi credentials and reboot into STA mode. AP fallback can be disabled with `ap disable`.

Reaching the WebUI

Mode	URL	Notes
STA	<code>http://k2so.local</code>	mDNS, works on most home networks. Some routers block mDNS -- use the IP from the serial log.
STA	<code>http://<ip></code>	Direct IP. Print on serial via <code>status</code> or <code>wifi show</code> .
AP	<code>http://192.168.4.1</code>	Always-on IP for the AP fallback.

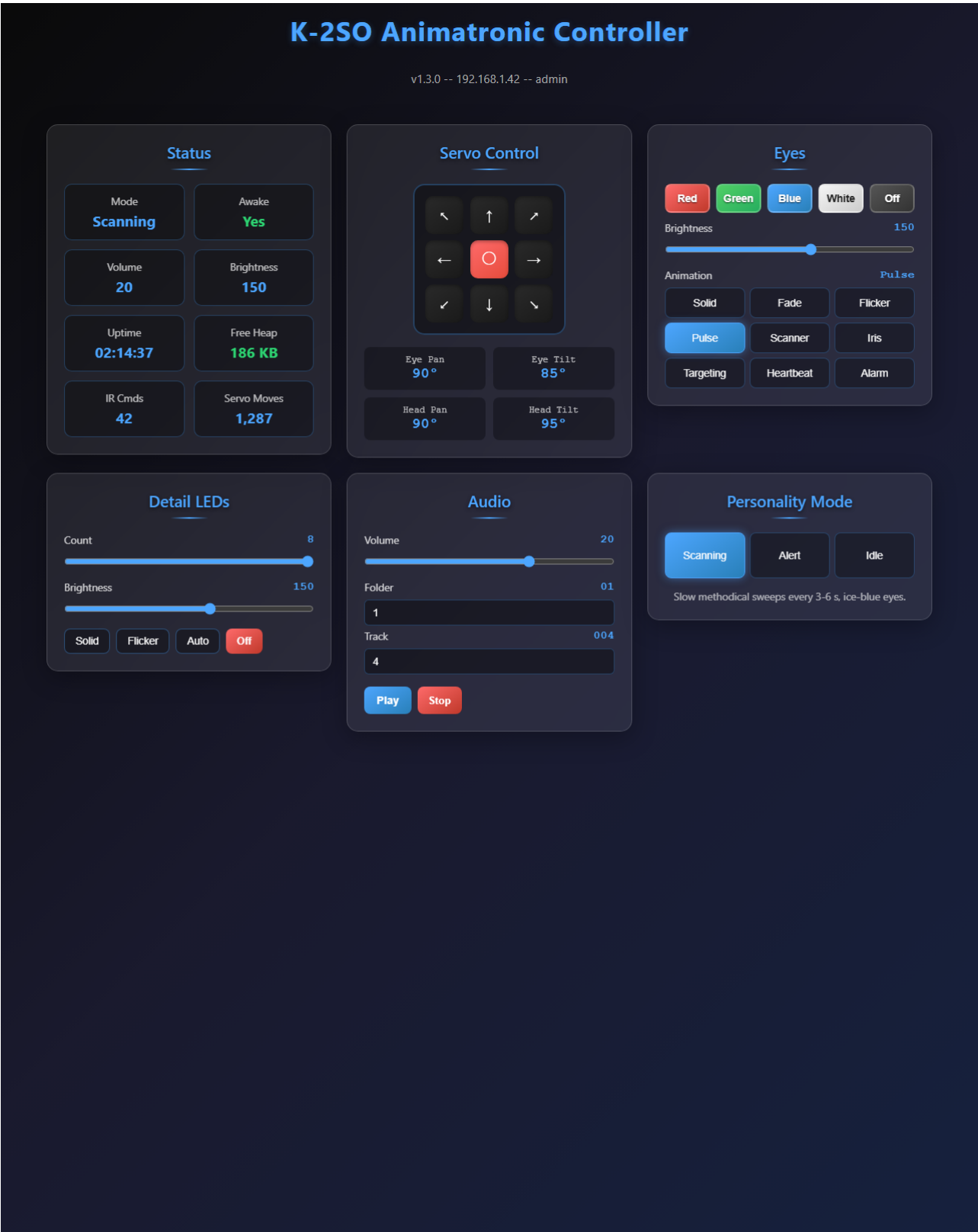
Authentication

All WebUI pages and trigger endpoints are protected by HTTP Basic Auth. Default credentials `admin / k2so2025`. Change them in `config.h` before exposing the droid to a network.

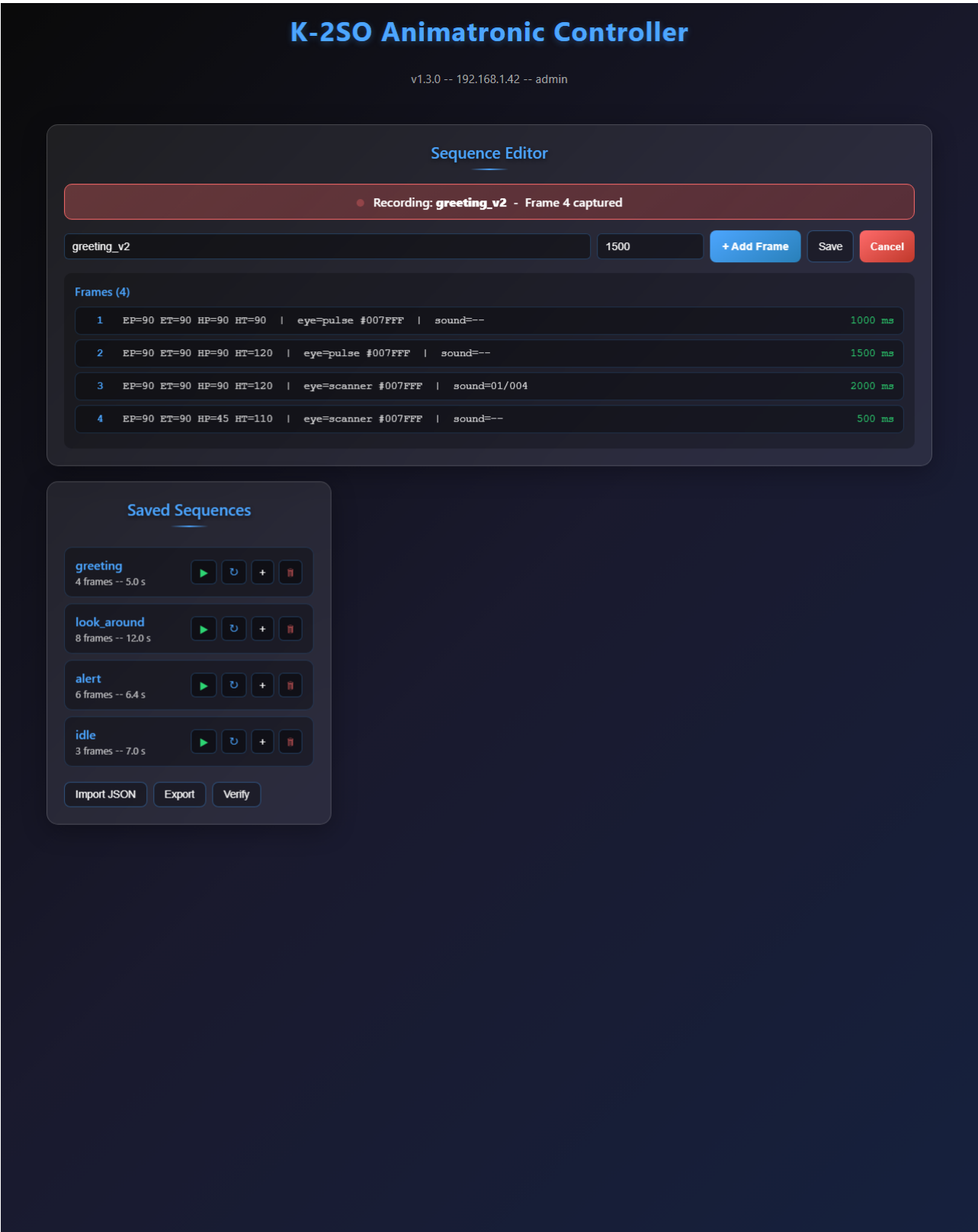
WebUI Tour

- **Status:** mode, awake state, volume, brightness, uptime, free heap, IR command counter, servo movement counter.
- **Servos (Gamepad):** two virtual sticks for Eye Pan/Tilt and Head Pan/Tilt; values clamp to the configured limits.
- **Eyes:** 14 animation modes as buttons; color picker; brightness slider; *red / green / blue / white / off* shortcuts.
- **Detail LEDs:** count selector (1-8), pattern selector, color picker, brightness slider, on/off, auto mode.
- **Audio:** volume slider, play folder/track form, stop button.

- **Mode:** Scanning / Alert / Idle radio buttons.
- **Sequences (NEW v1.3.0):** list of saved sequences, play/loop/stop/pause/resume/delete per item, "Record" form, frame editor.
- **Playlist (NEW v1.3.0):** add/clear/play/loop, current playlist view.
- **IR Mapping (NEW v1.3.0):** table of 21 buttons with dropdown to assign sequences.
- **Settings:** WiFi, AP, password, brightness defaults, eye hardware version (7-LED / 13-LED).



Web UI dashboard (v1.3.0).



Sequence editor with playlist controls.

Tip: the WebUI auto-refreshes the sequence list and playlist status. If changes do not appear, hard-reload with Ctrl+F5 -- some browsers cache aggressively.

Part II

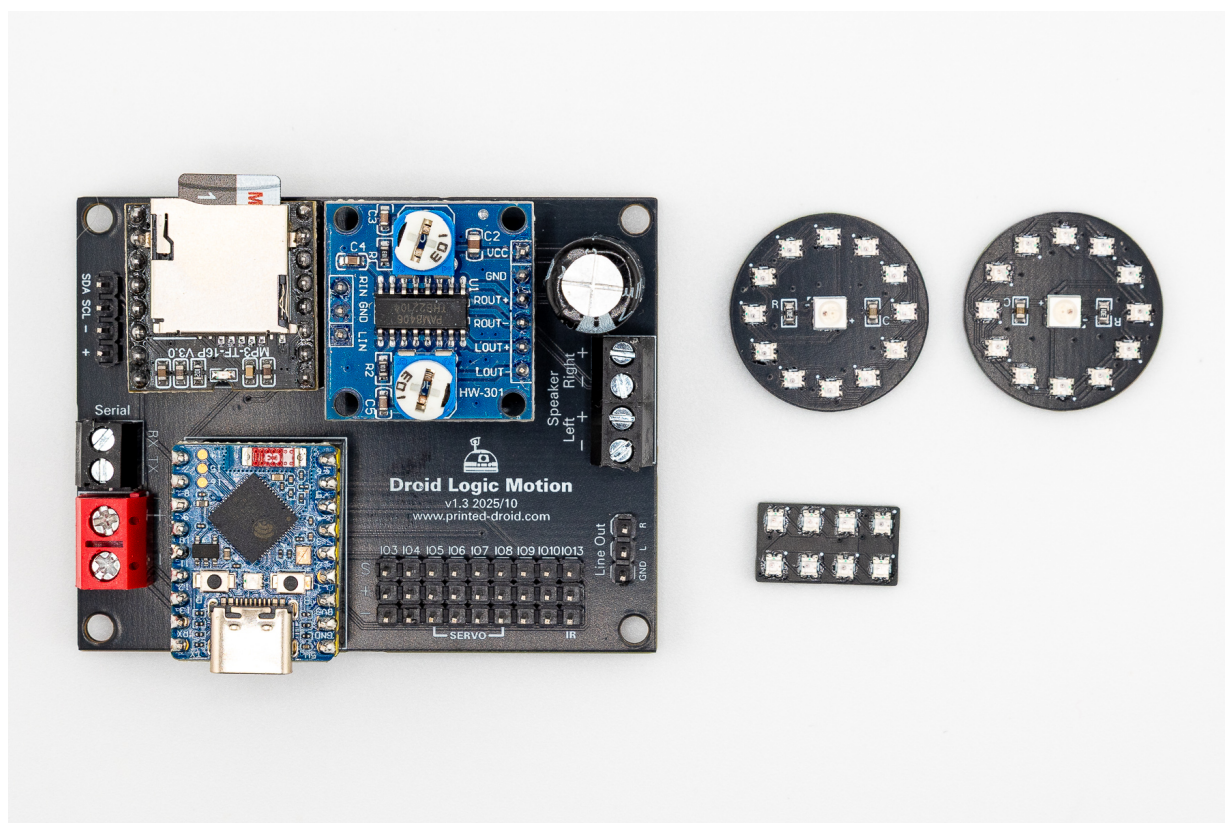
Hardware

Board & Pin Map

The recommended hardware is the **Droid Logic Motion v1.3**, which carries a Waveshare ESP32-S3-Zero plus the necessary level shifters, headers, and DFPlayer footprint. Standalone ESP32-S3-Zero builds on a breakout board work too -- the firmware does not care, as long as the GPIO assignments below match.

GPIO Pin Assignment (v1.3.0)

GPIO	Function	Direction	Notes
GP1	I2C SDA	I/O	Reserved for future expansion.
GP2	I2C SCL	OUT	Reserved for future expansion.
GP3	Left Eye NeoPixel	OUT	7 or 13 LEDs (runtime-switchable).
GP4	Right Eye NeoPixel	OUT	7 or 13 LEDs (runtime-switchable).
GP5	Eye Pan Servo	PWM	LEDC ch 0.
GP6	Eye Tilt Servo	PWM	LEDC ch 1.
GP7	Head Pan Servo	PWM	LEDC ch 2.
GP8	Head Tilt Servo	PWM	LEDC ch 3.
GP9	IR Receiver	IN	NEC/Sony codecs, learning mode.
GP10	Detail LED Strip	OUT	WS2812, 1-8 LEDs configurable.
GP11	DFPlayer TX	OUT	UART2 TX -> DFPlayer RX.
GP12	DFPlayer RX	IN	UART2 RX -< DFPlayer TX.
GP13	unused	--	Free since v1.2.x (was Detail-LED in v1.1).
GP21	Status LED	OUT	Single WS2812.



Droid Logic Motion v1.3 controller with eye boards and detail-LED board.

Connector Groups

The silkscreen on the controller labels every connector group where it sits. The table below maps each label to its electrical purpose and -- where applicable -- to the GPIO that handles it.

Silkscreen	Type	Purpose
SD Card	--	DFPlayer microSD slot.
SDA SCL +/-	4-pin header	I2C bus terminal (GP1 SDA, GP2 SCL, plus 5V/GND). Reserved for future expansion.
Serial RX TX	4-pin header	Optional debug header. USB-CDC works without this.
5V Input	2-pole screw	Main power input. 5V/3A min, 5V/5A recommended.
USBC	USB-C	Programming + power via ESP32-S3-Zero native USB CDC.
SERVO (SI/+/-)	3 cols x 9 rows	Servo headers labelled IO3 to IO13. Eye Pan = IO5, Eye Tilt = IO6, Head Pan = IO7, Head Tilt = IO8 in v1.3.0. IO10 carries the detail-LED data line. Other IOs are spare.
IR	3-pin header	IR receiver (signal/+/-) on GP9.
Speaker Left/Right +/-	4-pole screw	Amplifier output. 4-8 ohm full-range speakers.

Silkscreen	Type	Purpose
Line Out (L/R/GND)	3-pin header	Line-level output for an external amplifier. Bypasses the on-board PAM8406.

LED Boards (separate from controller)

Eye boards (left/right): 7-LED or 13-LED variants. Each board has a 3-pin header (Data In / +5V / GND). Wire *Data In* of the left eye to GP3, right eye to GP4. Power and ground from the same 5V rail as the controller.

Detail board: 8 LEDs in a 2x4 layout, 3-pin header. *Data In* connects to GP10 (IO10 row in the SERVO group, signal pin).

Wiring Notes

- **Common ground:** if servos and the ESP32-S3 are on different supplies, tie the grounds together. The DFPlayer must share ground with the ESP32-S3 or UART communication will be unreliable.
- **Servo signal level:** ESP32-S3 outputs 3.3V logic. Most hobby servos accept it; if a servo refuses to move, an external level shifter to 5V helps.
- **NeoPixel signal level:** WS2812 strips officially want 5V logic, but typically work from 3.3V over short cables (<50 cm). For longer runs, add a 74AHCT125 level shifter at the strip.
- **DFPlayer power-up order:** the DFPlayer expects ground first, then VCC. Hot-plugging it can fry the SD-card holder.
- **5V rail discipline:** short, thick power leads from PSU to the controller; do not daisy-chain power through several long thin wires.
- **GP13 is FREE since v1.2.x** -- do not wire anything there unless you understand the firmware does not look for a peripheral on this pin.

Tip: if you are designing a custom carrier board, leave GP1/GP2 (I2C) free for future IMU or sensor add-ons. Keep GP0, GP43-46, and the USB pins clear of strapping conflicts.

Power & PSU

Power is the single most common cause of erratic K-2SO behaviour. Brownouts during DFPlayer startup or NeoPixel flashes manifest as random reboots, dropped WiFi, missed IR codes, and silent DFPlayer. This chapter explains the budget and the workarounds.

Current Budget

Component	Quiescent	Peak
ESP32-S3-Zero	~80 mA	~250 mA (WiFi TX bursts)
DFPlayer Mini idle	~20 mA	~250 mA (track start)
4x SG90 servos idle	~40 mA	~600 mA each under load
2x 13-LED Eyes white max	~80 mA each	~700 mA each
Detail strip 8 LEDs white max	~50 mA	~480 mA
Status LED	negligible	~30 mA
Worst-case total	--	~3.5 A momentary, ~1.5 A sustained

5V/3A is the working minimum. A 5V/2A supply will brownout when the DFPlayer starts a track during a NeoPixel flash. A 5V/5A supply (USB-PD trigger module + step-down or a quality bench PSU) eliminates the problem.

Recommended Topology

For builds that go on a costume or into a display:

- **Single 5V/5A rail** from a quality PSU, fused at 5A, master switch on the input.
- **Servo supply with star-grounded** back to the PSU; do not chain servo grounds through the ESP32 board.
- **USB power-only** during programming -- once the firmware is on, switch to the dedicated PSU.

Battery Builds

For mobile builds, a 2S LiPo (7.4V nominal) into a 5V/5A buck converter is the simplest path. Avoid raw single-cell LiPo (3.7V) -- it sags below 4V quickly under servo load. A 2S 2200-3000 mAh pack runs the droid for 1-2 hours of moderate activity. Add a low-voltage cutoff (LVC) module to protect the cells.

Tip: the ESP32-S3 prints "Brownout detector was triggered" before resetting. If you see this in the serial log, your PSU is the suspect, not the firmware.

LED Hardware

The controller drives four independent NeoPixel chains: two eye rings, one detail strip, and one status LED. All four use the WS2812 protocol on RMT channels (the ESP32-S3 has 4 RMT TX channels, exactly enough). This chapter covers the hardware variants and the relevant defaults.

Eye NeoPixels: 7-LED vs 13-LED

Two eye-ring variants are supported and the choice is **runtime-configurable**, not compile-time:

Variant	Layout	Best for
7-LED	LEDs 0-6 in a single ring	Compact eye sockets, simpler builds.
13-LED (default)	LED 0 = center, LEDs 1-12 = surrounding ring	Iris and targeting modes use the center pixel; 13-LED gives all 14 modes their full effect.

Switch via Serial CLI: `led eye 7` or `led eye 13`. The setting is persisted in EEPROM. The boot sequence adapts to the selected variant.

Detail LED Strip

The detail strip on GP10 holds 1 to 8 WS2812 LEDs. Configure the count via `detail count N` (saved to EEPROM). Patterns include solid color, random flicker, and auto-mode. Brightness 0-255, default 150.

Status LED

A single WS2812 on GP21 visualizes system state. The states map to colors:

State	Color	Pattern	Meaning
BOOT	Blue	Blink	Initial boot, before WiFi.
WIFI_CONNECTING	Yellow	Blink	STA join attempt in progress.
WIFI_CONNECTED	Green	Solid 2s	STA up, IP acquired.
WIFI_DISCONNECTED	Red	Solid	STA dropped.
MODE_SCANNING	Ice blue	Pulse	Scanning personality active.
MODE_ALERT	Red	Pulse	Alert personality active.
MODE_IDLE	Amber	Pulse	Idle personality active.
IR_ACTIVITY	White	Flash	IR command received.
SERVO_ACTIVITY	Blue	Flash	Servo movement triggered.
AUDIO_ACTIVITY	Green	Flash	Audio playback triggered.
ERROR	Red	Fast blink	Generic error state.

State	Color	Pattern	Meaning
LEARNING_MODE	Purple	Blink	IR learning in progress.
CONFIG_MODE	Cyan	Pulse	Setup wizard / monitor mode.

Tip: the status LED is your first diagnostic tool. If WiFi never comes up (yellow blink for 30+ seconds), the AP fallback is starting. If the LED stays blue forever, the boot sequence got stuck -- check the serial log.

Part III

Core Features

Servo Calibration

Each build is mechanically slightly different. Calibration sets the four servos so the eyes point straight ahead at "center" and the limits respect the mechanical end-stops of your build. Skip this and the droid will either look cross-eyed or strip a gear -- both are bad.

Default Values

Servo	Center	Min	Max
Eye Pan	90	60	120
Eye Tilt	90	60	120
Head Pan	90	0	180
Head Tilt	90	0	180

Calibration Workflow

1. Mount the head loosely

The head should rest on the servo horns but not be load-bearing on its own weight. Fingers clear of all servo arms.

2. Center first

`servo eye center 90 90` (or pick your own). Eyes should look straight ahead. Adjust until they do, then save: `save`.

3. Set Eye limits

Pan the eyes left until the mechanism just stops being smooth -- that is your panMin. Same right for panMax. Tilt up = tiltMax, down = tiltMin. Then: `servo eye limits panMin panMax tiltMin tiltMax`.

4. Center head

`servo head center 90 90`.

5. Set Head limits

`servo head limits panMin panMax tiltMin tiltMax`. Min must be less than max -- the firmware rejects `min >= max` with a clear error.

6. Verify

`servo test all` sweeps each servo from min to max. If anything binds, scratches, or tries to move past a stop, fix the limits and retest.

7. Save as a profile

Once happy: `profile save MyBuild`. You can store up to 5 named profiles and switch between them.

Min < Max enforcement (NEW v1.2.4): the firmware rejects servo limit commands where $\text{min} \geq \text{max}$. Earlier versions would silently accept inverted limits, causing erratic motion.

Profiles

Profiles bundle: servo centers and limits, mode, volume, eye brightness, scan/alert timings, and mode colors. Five slots (0-4). `profile list`, `profile save name`, `profile load N`, `profile delete N`, `profile show N`. Saving rejects when all 5 slots are full -- delete one first (NEW v1.2.4 guard).

Eye Animations

The eye-animation engine has 14 modes. Switch with `led mode <name>` or via the WebUI. Color, brightness, and animation speed are independent of the mode and apply whichever mode is active.

Animation Modes

Mode	Description	Best with
<code>solid_color</code>	Static color, no animation.	7 / 13 LED
<code>fade_off</code>	Fade from current color to off.	7 / 13 LED
<code>fade_color</code>	Smooth fade between two colors.	7 / 13 LED
<code>flicker</code>	Random brightness flicker.	7 / 13 LED
<code>pulse</code>	Smooth brightness pulse.	7 / 13 LED
<code>scanner</code>	Scanner sweep across all LEDs.	7 / 13 LED
<code>iris</code>	Ring pulses, center stays static.	13 LED only
<code>targeting</code>	Ring rotates, center blinks.	13 LED only
<code>ring_scanner</code>	Scanner sweep in ring only.	13 LED only
<code>spiral</code>	Spiral effect from outside in.	7 / 13 LED
<code>focus</code>	Ring blinks, center stays on.	13 LED only
<code>radar</code>	Radar sweep in the ring.	13 LED only
<code>heartbeat</code>	Synchronized heartbeat pulse on both eyes.	7 / 13 LED
<code>alarm</code>	Synchronized alarm flash on both eyes.	7 / 13 LED

Color & Brightness

`led color R G B` sets the active color (0-255 each). `led brightness 0-255` sets eye brightness. The WebUI offers the classic shortcuts *red / green / blue / white / off* as buttons. Brightness is also persistable per profile and survives reboot.

[Image placeholder] `eye_modes_overview`

Eye animation modes (13-LED variant).

(Image will be supplied later. The PDF builds without it.)

Tip: the IRIS, TARGETING, RING_SCANNER, FOCUS, and RADAR modes use the center pixel and look incomplete on a 7-LED ring. Stick to the 7-LED-friendly modes (solid, fade_*, flicker, pulse, scanner, spiral, heartbeat, alarm) if your hardware is 7-LED.

Audio (DFPlayer Mini)

Audio is provided by a **DFPlayer Mini** connected via UART (GP11 TX, GP12 RX, 9600 baud). It plays MP3 files from a microSD card. The Printed-Droid bundle includes 40+ K-2SO-style sound clips in the K-2SO Audio Files/ folder of the project.

SD Card Preparation

- **Format:** FAT32, max 32 GB.
- **Folder layout:** /01/, /02/, ... -- two-digit folder names, max 99.
- **File naming:** 001.mp3, 002.mp3, ... -- three-digit, max 999 per folder.
- **Encoding:** standard MP3, mono or stereo, sample rates 8-48 kHz.
- **Hidden files:** macOS leaves `._*` resource forks. Clean them with `dot_clean` on macOS or by mounting on Linux/Windows.

CLI Reference

Command	Effect
<code>sound show</code>	Show current volume and audio status.
<code>sound volume 0-30</code>	Set DFPlayer volume (0=mute, 30=max).
<code>sound play <track></code>	Play track from root.
<code>sound folder <f> <t></code>	Play track <i>t</i> from folder <i>f</i> .
<code>sound stop</code>	Stop playback.

Default Volume & Auto-Mode

Default volume after first boot is 20 (out of 30). The personality engine plays random K-2SO lines from a configured folder when in Scanning or Alert mode, with a configurable pause range (8-20 seconds default). Adjust pause via `timing sound <min> <max>`.

DFPlayer-not-ready warning: if the SD card is missing, wrongly formatted, or the DFPlayer chip is the older clone variant, you will see "DFPlayer not ready" in the serial log. Check the SD first; if that is fine, try a different DFPlayer module.

IR Remote Setup

The controller can learn any infrared remote that uses NEC, Sony, or similar standard codecs. Up to 21 buttons can be programmed. Once learned, each button can be assigned to a built-in action or (NEW v1.3.0) to a saved sequence.

Learning Mode Workflow

1. Enter learning mode

Type `learn` in the serial monitor. The status LED switches to purple blink.

2. Press a button on the remote

The controller captures the IR code and prompts you to name the button.

3. Name the button

Use a short label like `play`, `stop`, `7`. Hit enter.

4. Repeat for each button

You can stop at any point with `exit` or `normal`.

5. Verify

`show` lists all programmed buttons and their codes.

Learning mode blocks other tasks. While the controller is in `learn` mode, servo updates and animations are paused. Do not learn while the droid is on a costume.

IR-Related Commands

Command	Effect
<code>learn</code>	Enter learning mode (purple LED).
<code>scan</code>	Print live IR codes for diagnostics.
<code>show</code>	List all configured buttons.
<code>clear</code>	Wipe all IR mappings (with confirm).
<code>ir on / ir off</code>	Enable / disable IR receiver.

[Image placeholder] `ir_remote_layout`
Typical 21-button learning remote.
(Image will be supplied later. The PDF builds without it.)

Tip: if the same physical remote sends two slightly different codes for the same button (some remotes do this with a repeat-bit), you only need to learn it once. The controller treats both as the same button.

Personality Modes

When idle, the droid runs autonomously in one of three personality modes. The mode controls how often the eyes move, how big the moves are, how often sounds play, and what color the eyes rest on. Switch between modes via mode <name> or the WebUI.

Mode	Character	Movement	Default Color
scanning	Methodical, observant.	Slow eye sweeps every 3-6 s, large angle.	Ice blue
alert	Quick, reactive.	Fast eye darts every 0.5-1.5 s, small angle.	Red
idle	Power saving, almost still.	Minimal motion, occasional twitch.	Amber

Tuning Mode Behaviour

timing show displays current values. Edit with:

Command	Range	Effect
timing scan move <min> <max>	20-200 ms	Scanning eye-move duration range.
timing scan wait <min> <max>	500-30000 ms	Scanning pause between moves.
timing alert move <min> <max>	5-100 ms	Alert eye-move duration range.
timing alert wait <min> <max>	100-5000 ms	Alert pause between moves.
timing sound <min> <max>	1000-300000 ms	Random sound interval (both modes).

Wake / Sleep Behaviour

After 60 minutes of inactivity (no IR, no web request, no servo command), the droid enters a low-activity state ("not awake"). Any input wakes it back up. The voice trigger /trigger/sleep forces sleep, /trigger/wakeup forces wake.

Sequence Recording (NEW v1.3.0)

The Sequence Recording engine -- new in v1.3.0 (promoted from v1.2.5 beta) -- lets you capture a series of "frames" (discrete poses with audio cues) and replay them as a smooth motion. Sequences are stored in LittleFS as JSON files, persist across reboots, and can be chained into playlists (next chapter) or triggered by IR buttons (chapter after).

What is Captured per Frame

- 4x servo positions (Eye Pan/Tilt, Head Pan/Tilt) at the moment the frame is added.
- Eye animation mode, color, brightness.
- Detail strip pattern, color, brightness.
- Optional sound trigger: folder + track + volume.
- Frame duration in milliseconds (1-60000 ms).

Capacity

Limit	Value	Notes
Frames per sequence	200	Hard cap; recording stops when reached.
Sequence name length	1-31 chars	Allowed: A-Z, a-z, 0-9, underscore, hyphen.
Storage available	~1.5-2 MB	Of the 4MB ESP32-S3 flash, partitioned for LittleFS.
Per-frame size	~80 B JSON / ~24 B RAM	JSON overhead vs in-memory struct.

Recording Workflow (Serial CLI)

1. Start a recording

`seq new "MyName" -- name follows the rules above.`

2. Pose the droid

Use the WebUI gamepad or any servo / led / detail / sound command to set the desired state.

3. Capture the frame

`seq frame <duration_ms> -- e.g. 1500 for 1.5 seconds.`

4. Repeat

Pose, capture, pose, capture. Up to 200 frames.

5. Save

`seq save` writes the JSON to LittleFS at `/sequences/<name>.seq`.

6. Cancel anytime

`seq cancel` drops the in-progress recording without saving.

Playback Commands

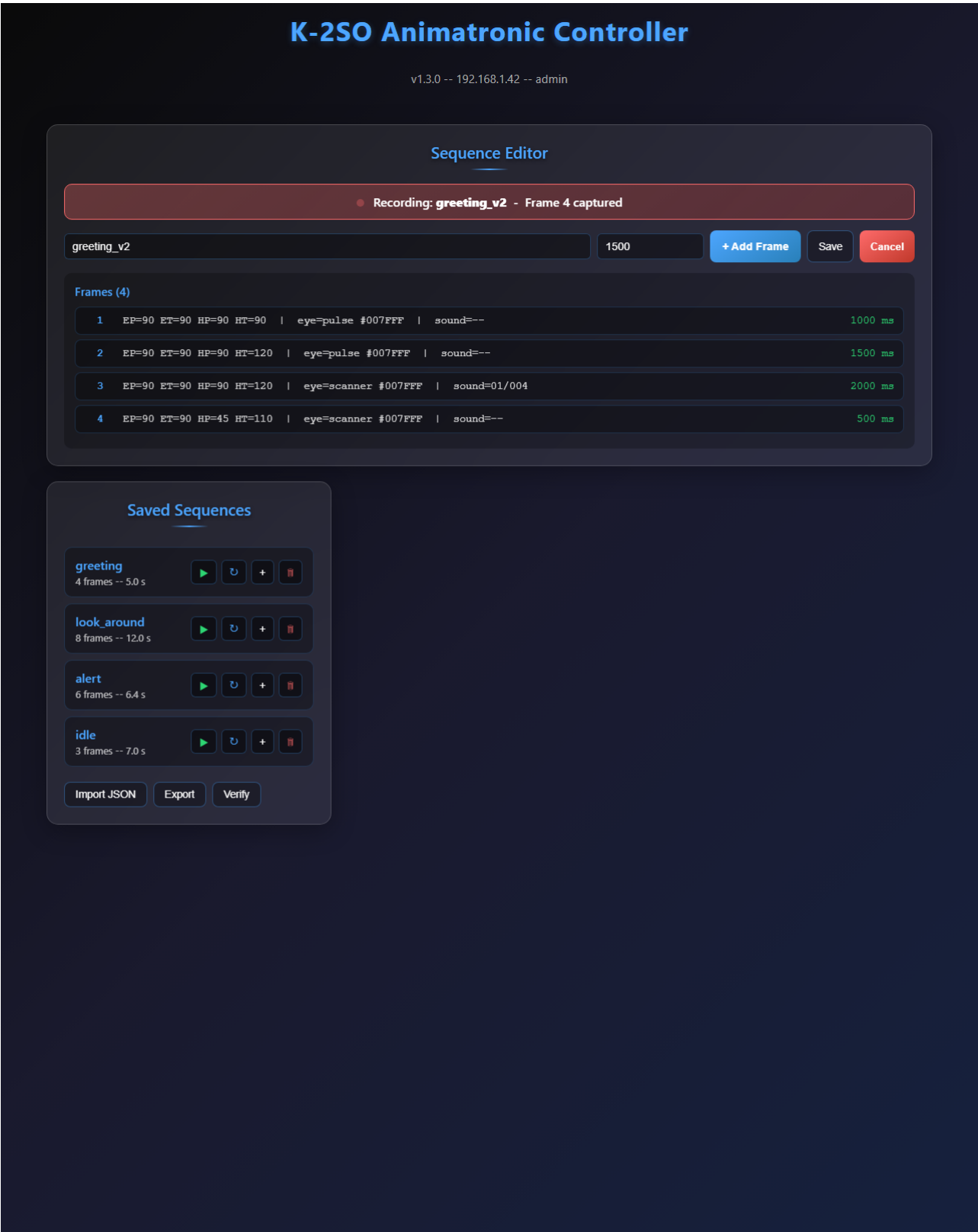
Command	Effect
<code>seq play "name"</code>	Play once.
<code>seq loop "name"</code>	Play in a loop until stopped.
<code>seq stop</code>	Stop active playback.
<code>seq list</code>	List all saved sequences (max 20 shown).
<code>seq info "name"</code>	Show metadata: frame count, total duration.
<code>seq rename "old" "new"</code>	Rename a sequence.
<code>seq delete "name"</code>	Delete with confirm prompt.
<code>seq status</code>	Show current recording / playback state.
<code>seq format</code>	Reformat LittleFS (with confirm). NEW v1.3.0 -- replaces silent auto-format.
<code>seq verify "name"</code>	NEW v1.3.0 follow-up -- check JSON integrity of a saved sequence.
<code>seq verify all</code>	NEW v1.3.0 -- bulk-verify every saved sequence in one pass (good before a show).
<code>seq export "name"</code>	NEW v1.3.0 follow-up -- print the sequence JSON to serial for backup.
<code>seq import</code>	NEW v1.3.0 follow-up -- paste a sequence JSON to import it.
<code>seq duplicate "old" "new"</code>	NEW v1.3.0 -- copy an existing sequence to a new name without re-recording.
<code>seq stats</code>	NEW v1.3.0 -- LittleFS / sequences / playlists overview (free space, file counts).

Crash-Safe Save (NEW v1.3.0 follow-up)

As of the v1.3.0 follow-up, seq save uses a temp/backup/replace flow on LittleFS: the new content is first written to <name>.tmp, the existing target is renamed to <name>.bak, the temp is promoted to the real target only after the JSON write succeeds, and the backup is restored on any failure. Net effect: you cannot lose the last good copy of a sequence by saving over it.

Pause and Resume (Web UI Only)

Pause and Resume are exposed only via the WebUI buttons (and the HTTP endpoints /seq/pause and /seq/resume). There are no serial commands -- by design, since pausing a sequence at exactly the right moment is hard on serial.



WebUI sequence editor with frame timeline.

Tip on durations: 300-800 ms for fast moves, 1000-2000 ms for normal moves, 2000-4000 ms for slow / dramatic moves, 500-1000 ms for pauses. Multi-frame sequences feel much more natural than one giant 60-second recording.

Playlist Chaining (NEW v1.3.0)

A playlist chains up to 10 saved sequences and plays them back-to-back. Use it to compose longer performances from short, reusable building blocks: *greeting* -> *look_around* -> *scan* -> *idle*.

Building a Playlist

Command	Effect
<code>seq playlist add "name"</code>	Append to current playlist (max 10).
<code>seq playlist clear</code>	Empty the playlist.
<code>seq playlist</code>	Show current playlist (no args).
<code>seq playlist play</code>	Play once through.
<code>seq playlist loop</code>	Play in loop until stopped.
<code>seq playlist remove <n></code>	NEW v1.3.0 follow-up -- remove playlist entry n (1-based).
<code>seq playlist move <from> <to></code>	NEW v1.3.0 follow-up -- reorder playlist (1-based indices).
<code>seq playlist save "name"</code>	NEW v1.3.0 -- persist current playlist as a named .pls file in LittleFS.
<code>seq playlist load "name"</code>	NEW v1.3.0 -- restore a saved named playlist (load rejects missing referenced sequences).
<code>seq playlist list</code>	NEW v1.3.0 -- list all named playlists from /playlists/.

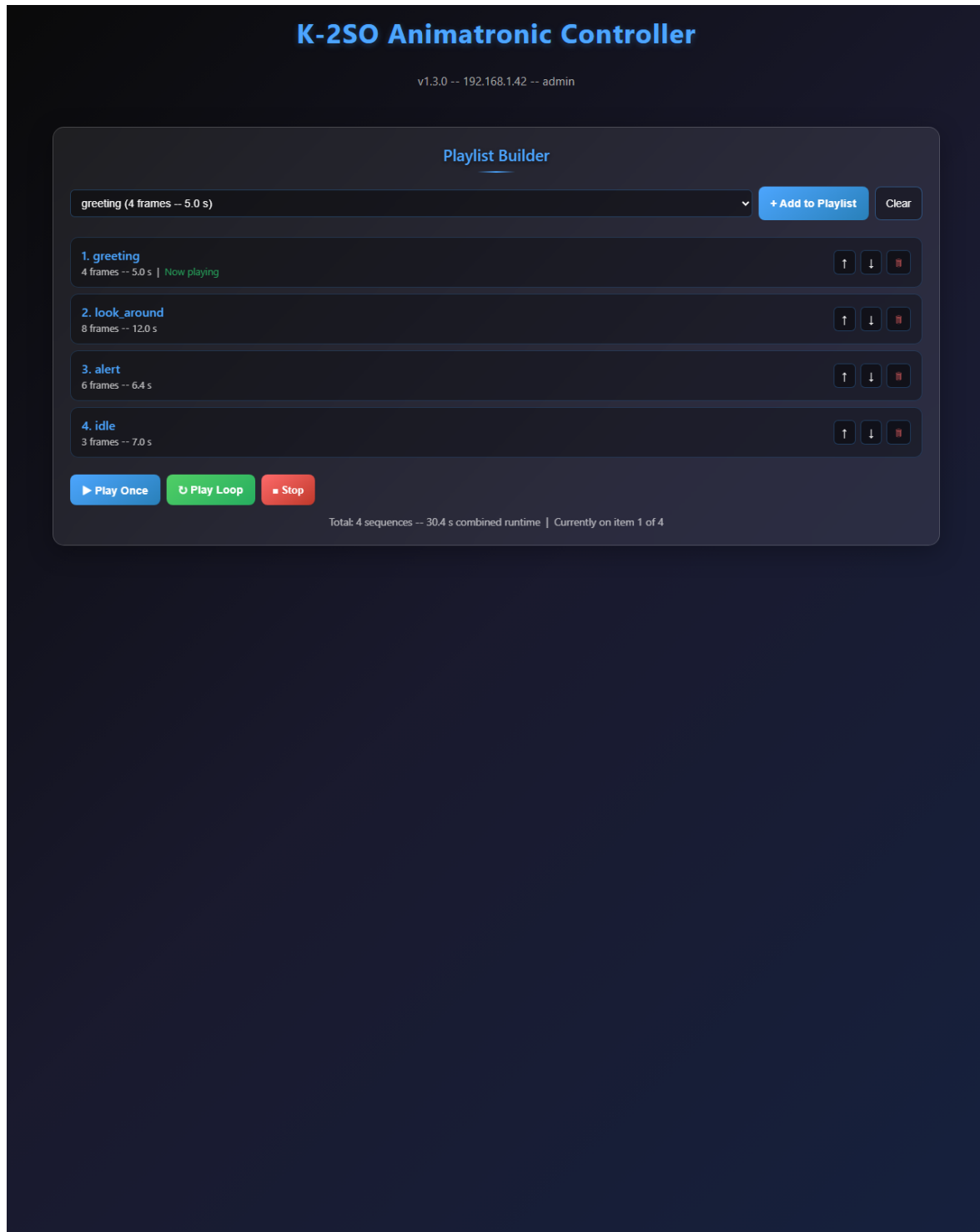
Behaviour and Robustness

- **Atomic transitions:** when one sequence ends and the next is loaded, the playlist preserves its state across the handoff. Earlier (v1.2.5 beta) versions dropped the playlist after the first transition -- fixed in v1.3.0.
- **Failure recovery:** if the next sequence cannot load (file deleted, JSON corrupted, out-of-memory), the playlist stops cleanly and the droid returns to *Idle* state instead of leaving a half-playing zombie.
- **Persistent named playlists (NEW v1.3.0):** the active playlist is still RAM-only, but you can save it under a name with `seq playlist save "myshow"`. Saved playlists live in `/playlists/myshow.pls` on LittleFS and survive a power-cycle. Restore with `seq playlist load "myshow"`; load rejects missing referenced sequences with a clear error.

Example Performance

A 30-second compound performance:

```
seq playlist clear
seq playlist add "greeting" (5 s)
seq playlist add "look_around" (10 s)
seq playlist add "alert" (8 s)
seq playlist add "idle" (7 s)
seq playlist play
```



Playlist builder in the Web UI.

Tip: short, reusable sequences are better than one giant scripted performance. You can mix and match playlists for different convention scenes without re-recording.

IR-to-Sequence Mapping (NEW v1.3.0)

IR-to-Sequence mapping (new in v1.3.0) lets you assign any of the 21 learned IR buttons to a saved sequence. Press the button on the remote, the corresponding sequence plays. Mappings live in EEPROM and survive reboots.

Prerequisites

- IR remote already learned (see IR Remote Setup chapter).
- At least one saved sequence (see Sequence Recording chapter).
- Buttons should have meaningful names -- mapping is by name, not by index.

Mapping Commands

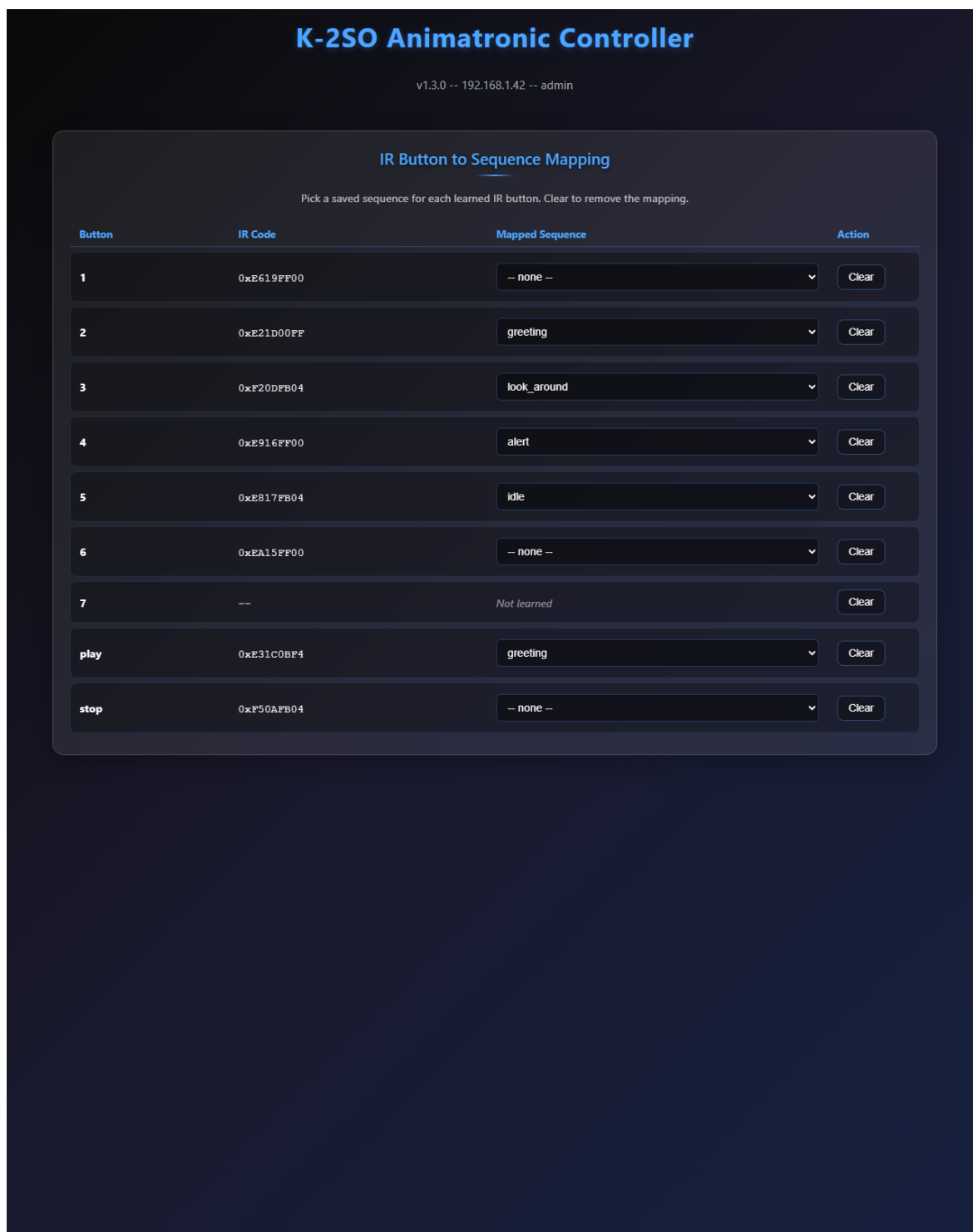
Command	Effect
seq map <button> "name"	Assign sequence "name" to the named button.
seq map <button> clear	Remove the mapping for one button.
seq map	Show all current mappings.

Example

You have learned IR buttons named 7, 8, 9 and recorded sequences greeting, alert, look_around:

```
seq map 7 "greeting"  
seq map 8 "alert"  
seq map 9 "look_around"  
seq map # show resulting table
```

What happens at runtime: the controller decodes IR → finds the matching button name → if a sequence is mapped, plays it once (non-loop). The serial log prints IR triggering sequence: greeting. If no sequence is mapped, the button keeps its built-in action (or none, if not bound).



IR-to-Sequence mapping table in the WebUI.

Tip: mapping the playlist-play button (e.g. button "0") to a placeholder sequence does not start the playlist -- it always starts a single sequence. To launch a playlist via IR, trigger it indirectly through a Voice Assistant routine that hits the `/seq/playlist/play` endpoint.

Part IV

Reference & Appendices

Serial CLI Reference

Complete reference for the serial command-line interface (115200 baud). Commands are case-insensitive on the root verb and case-sensitive on names (sequences, profiles). All confirmation prompts have a 30-second timeout (NEW v1.2.4) -- a stuck droid will recover on its own if the operator walks away mid-prompt.

Top-Level Commands

Command	Purpose
help	Print command summary.
status	Show overall system state.
show	List learned IR buttons.
save	Persist current settings to EEPROM.
config	Show full configuration dump.
default	Reset config to factory defaults (with confirm).
clear	Wipe IR mappings (with confirm).
reset	Reboot the ESP32.
monitor	Live status loop until any key.
test	Hardware self-test menu.
demo	Run demo sequence.
exit / normal	Leave a special mode (learn / monitor / setup).
backup	Dump full configuration as hex to serial.
restore	Read hex back; confirms checksum (NEW v1.2.4).
learn	Enter IR learning mode.
scan	Print live IR codes.
ir on / ir off	Enable / disable IR.
mode <name>	Personality: scanning / alert / idle.

Hierarchical Commands

These have subcommands. Type the verb without arguments to see the help.

Group	Subcommands
servo	show / eye center / eye limits / head center / head limits / test
led	show / brightness / color / mode / eye / status / test
detail	show / count / brightness / color / pattern / on / off / auto / test
sound	show / volume / play / folder / stop
timing	show / scan move / scan wait / alert move / alert wait / sound
profile	list / save / load / delete / show
wifi	show / set / reset / reconnect
ap	show / set / reset / enable / disable / start
seq	new / frame / save / cancel / play / loop / stop / list / info / delete / rename / playlist (add / clear / play / loop / remove / move / save / load / list) / map / status / format / verify / verify all / export / import / duplicate / stats

Common Quick Commands

What	Command
Center all servos	<code>servo eye center 90 90 + servo head center 90 90</code>
Hardware self-test	<code>servo test all</code>
Set eyes blue, pulse	<code>led color 0 0 255 + led mode pulse</code>
Play sound 1 from folder 1	<code>sound folder 1 1</code>
Save current settings	<code>save</code>
Show IP and WiFi state	<code>wifi show</code>
Backup config	<code>backup -- copy the hex output</code>

Voice Assistants

The controller exposes 10 HTTP trigger endpoints under `/trigger/*`. Any voice assistant or home-automation system that can issue an HTTP GET with Basic Auth (Google Home, Alexa, Siri Shortcuts, Home Assistant, IFTTT, Node-RED) can drive the droid.

Trigger Endpoints

Endpoint	Voice phrase	Effect
<code>/trigger/wakeup</code>	"K2SO wake up"	Scanning mode + blue scanner eyes + sound.
<code>/trigger/standby</code>	"K2SO standby"	Quiet mode, gentle blue glow, no movement.
<code>/trigger/sleep</code>	"K2SO sleep"	Eyes off, sound off, motion off.
<code>/trigger/demo</code>	"K2SO demo"	Run a full demo sequence.
<code>/trigger/speak</code>	"K2SO speak"	Play a random voice line.
<code>/trigger/alert</code>	"K2SO alert mode"	Switch to Alert personality, amber eyes.
<code>/trigger/scanner</code>	"K2SO scanner eyes"	Classic scanner eye animation.
<code>/trigger/alarm</code>	"K2SO alarm"	Red flashing alarm with sound.
<code>/trigger/center</code>	"K2SO center"	Center all servos.
<code>/trigger/patrol</code>	"K2SO patrol"	Patrol mode with movements and sounds.

Setup Example: Google Home / IFTTT

1. In IFTTT, create an Applet with trigger "Google Assistant -- Say a simple phrase". Phrase: *K2SO wake up*.
2. Action: "Webhooks -- Make a web request". URL: `http://<your-droid-ip>/trigger/wakeup`. Method: GET. Add Basic Auth header: Authorization: Basic YWRtaW46azJzbzIwMjU= (replace with your own admin:password base64-encoded).
3. Save. Say the phrase. The droid responds within ~1 s.

Curl / Manual Testing

```
curl -u admin:k2so2025 http://192.168.1.100/trigger/wakeup
```

Authentication is mandatory. All trigger endpoints require valid HTTP Basic Auth. If you set `WEB_AUTH_USER` to an empty string in `config.h`, auth is disabled -- not recommended for production.

Configuration & Backup

The controller stores its configuration in a 2048-byte EEPROM region (NVS-backed flash on ESP32-S3). The layout is identified by a magic byte (0xC0) and a version number. v1.3.0 uses version 2 (with the new IR-button-to-sequence mapping field); v1.2.x used version 1. Migration is automatic (next chapter).

What Is Stored

Group	Content
Identification	magic byte (0xC0), version (2), write counter.
IR Buttons	21 entries: name, IR code, configured flag, sequence-name (NEW v1.3.0).
Servo calibration	Eye Pan/Tilt center+min+max, Head Pan/Tilt center+min+max.
WiFi	STA SSID, STA password, configured flag.
AP	AP SSID, AP password, configured flag, enabled flag.
LEDs	Eye brightness, LED effect speed, eye hardware version (7 / 13 LED), status LED brightness, status LED enabled.
Timing	Scan / Alert / Sound min and max ranges.
Profiles	5 profile slots, each with mode, volume, brightness, servo centers, timings, mode colors.
Integrity	CRC-style checksum at the end.

Backup via Serial

backup dumps the entire 2048-byte struct as ASCII hex. Copy the output into a text file. The output looks like C0 02 00 00 00 . . . 7F -- bytes separated by spaces, ~4 KB of text.

Restore via Serial

restore prompts for confirmation, then accepts pasted hex. The firmware verifies **magic byte + version + checksum** before writing. NEW v1.2.4: invalid checksum is rejected with a clear error -- earlier versions silently accepted garbage. **NEW v1.3.0 follow-up: dual-format restore.** Both v2 hex (current) and legacy v1 hex (from v1.2.3 / v1.2.4 backups) are accepted directly. The legacy path runs the same migration helper used at boot, so settings land in the v2 layout with no data loss. Multi-line pasted hex is handled correctly -- whitespace and line breaks within the block are tolerated.

Profiles vs Backup

Profiles save a slice of the configuration -- mode, volume, brightness, servo centers, timings, mode colors -- to one of 5 named slots, on the same EEPROM as the rest. They are meant for quick "small / large droid" or "scanning / alert" presets, not as full backups.

Backup/Restore dumps the full 2048-byte struct, including all profiles, IR mappings, WiFi credentials, etc. Use it before flashing major upgrades or after a careful calibration session.

The backup hex contains your WiFi password in plaintext. Treat the backup file as you treat any password file -- do not commit it to git, do not paste it in public.

Migration v1.2.x to v1.3.0

v1.3.0 introduces an EEPROM layout change (the IR-button struct grew by 32 bytes per button to hold the sequence-name field). Without migration, an existing v1.2.3 / v1.2.4 controller would lose all settings on first boot of v1.3.0 -- WiFi password, IR mappings, servo calibration, profiles, everything. This was unacceptable, so v1.3.0 includes **automatic EEPROM migration**.

What Happens on First Boot After Upgrade

1. The firmware reads the EEPROM and inspects the magic byte (0xC0) and version field.
2. If magic = 0xC0 and version = 1, the migration path runs: the firmware reads the EEPROM as a v1 layout, validates the v1 checksum, copies all fields into the new v2 layout, initializes the new IR-button sequence-name fields to empty, recomputes the v2 checksum, and writes the result.
3. The serial log shows: Config migrated from v1 (v1.2.3/v1.2.4) to v2 (v1.3.0). All settings preserved.
4. Subsequent boots see version = 2 and run the normal load path.
5. If the v1 checksum was invalid (corrupt prior install), the migration falls back to factory defaults rather than corrupting v2.

Preserved Fields

- WiFi STA SSID and password.
- AP SSID, AP password, AP enabled flag.
- 21 IR button mappings (name + code + configured flag).
- Servo calibration (Eye Pan/Tilt and Head Pan/Tilt centers + min/max).
- All 5 profile slots (mode, volume, brightness, servo centers, timings, mode colors).
- LED settings: brightness, effect speed, eye hardware version (7-LED / 13-LED), status LED settings.
- Timing ranges (scan move/wait, alert move/wait, sound pauses, boot sequence delay).
- Current personality mode and volume.

New / Reset Fields

- IR-button sequence-name fields are initialized to empty (you assign them via seq map).
- Sequences (LittleFS) are not affected -- they live on a separate flash partition.

Recommended Upgrade Procedure

1. Make a serial backup on v1.2.4: backup, save the hex output.
2. Flash v1.3.0 firmware over USB.
3. Open Serial Monitor at 115200; the boot log should print the migration message.
4. Verify: `wifi show`, `show (IR)`, `servo show`, `profile list` -- all should match what you had on v1.2.4.
5. If something is wrong: re-flash v1.2.4, restore the hex backup, retry. If the migration still fails on retry, the v1 EEPROM was already corrupt -- fall back to factory defaults and reconfigure.

Downgrade is not supported. Once v1.3.0 has written version 2 to EEPROM, flashing v1.2.4 back will see an unknown version and reset to defaults. If you want to roll back, restore from your serial backup after flashing v1.2.4.

Troubleshooting

Most issues fall into a small set of categories: power, libraries, WiFi, RMT/IR coexistence, and SD card. The serial log at 115200 baud is your single best diagnostic tool -- the firmware is verbose by design.

Symptom-to-Fix Table

Symptom	Likely cause	Fix
Random reboots, "Brownout detector was triggered" in log	PSU undersized.	5V/3A min, 5V/5A recommended. Short, thick power leads from PSU.
Servos jitter, buzz, do not move smoothly	Brownout, wrong PWM channel, or limit min >= max.	Check PSU first. Then: servo show, verify min < max. Re-run servo test all.
IR commands missed when NeoPixel is animating	Library version mismatch (NeoPixel + IRremote on RMT).	Pin to verified versions (see Sketch Upload chapter). Update both together.
DFPlayer silent, "DFPlayer not ready" in log	SD card layout wrong, hidden files, or older clone DFPlayer.	FAT32, /01/001.mp3 layout, dot_clean on macOS. Try a different DFPlayer module.
WiFi never connects, status LED stays yellow	SSID/password wrong, or AP mDNS blocked.	wifi reset + wifi set Use IP from log. AP fallback after 30 s.
Web UI 401 Unauthorized	Default password changed or wrong creds.	Recheck WEB_AUTH_USER / WEB_AUTH_PASS in config.h.
Sequence loads but plays only one frame, then stops	Truncated JSON file or out-of-memory.	seq info name shows actual frame count. If too small, re-record. Free heap visible in /status.
Playlist plays first sequence, then stops	Was a v1.2.5 beta bug -- fixed in v1.3.0.	Upgrade to v1.3.0. If you are already on v1.3.0, the next sequence may be missing or corrupt -- check seq list.
"Invalid sequence name" error	Sequence name has illegal chars.	Allowed: A-Z, a-z, 0-9, _, -. Length 1-31 chars.
IR learn mode never returns	30 s timeout reached.	exit in serial. Re-enter learn. Send IR within 30 s.
EEPROM seems empty / settings reset themselves	Migration fallback after a corrupt v1 EEPROM, or NVS partition was erased.	Restore from serial backup. If no backup: factory defaults and reconfigure once.

Tip: when in doubt, run `monitor`. It prints status every second until you press a key. Watch for free-heap drops, IR command counter changes, mode transitions, and brownout indicators.

Version History

This chapter summarizes the released firmware versions. v1.3.0 is the current Stable Candidate; v1.2.4 is the previous Stable; v1.1.0 is archive. Older versions are documented superficially -- they are not recommended for new builds.

v1.3.0 -- Promotion Candidate (2026-04)

Highlights: promotes the v1.2.5 beta features to stable. Frame-based Sequence Recording (200 frames max), Playlist Chaining (10 sequences), IR-button-to-sequence Mapping. Automatic v1 -> v2 EEPROM migration preserves all existing data.

Critical fixes (over v1.2.5 beta): playlist chaining no longer stops after the first transition; truncated sequence JSON cannot drive servos with uninitialized memory; sequence names are sanitized against path traversal; LittleFS no longer auto-formats on mount failure; JSON document allocation guarded against OOM.

Quality (over v1.2.5 beta): failed .seq writes clean up partial files; sequence-related HTTP endpoints return real status codes (4xx) on errors instead of HTTP 200 with "ERROR" body; AP-default-password serial output uses the actual macro instead of a stale literal; restoreFromSerial log message fixed.

Follow-up additions: crash-safe sequence save (temp / backup / replace flow); legacy serial restore now accepts v1 hex backups directly (dual-format restore); new sequence manager methods `verifySequence`, `exportSequenceJson`, `importSequenceJson`, `playlistRemove`, `playlistMove`; new CLI commands `seq verify / export / import` and `seq playlist remove / move`; sequence/IR endpoints standardized on {ok, message} JSON envelope. WebUI consumes the new envelope and adds a sequence-import button. `getSequenceInfo()` no longer deletes a file on local OOM.

Feature Round (final v1.3.0 additions): `seq duplicate "old" "new"` for quick variants without re-recording; `seq verify all` for bulk pre-show checks; `seq stats` for LittleFS / sequence / playlist overview; **persistent named playlists** via `seq playlist save / load / list` (stored as .pls files in /playlists/); new HTTP endpoints `/seq/duplicate`, `/seq/verify-all`, `/seq/stats`, `/seq/playlist/save`, `/seq/playlist/load`, `/seq/playlist/list`; `/seq/map/list` now returns all configured IR buttons (mapped and unmapped) so the WebUI can assign empty rows directly; WebUI buttons added for *Verify All*, *Stats*, per-sequence Copy / Export / Verify, playlist Save / Load / Move / Remove, and IR-mapping re-assign.

Status: compile passes (~60% flash (No OTA partition), ~16% RAM). Hardware verification in progress.

v1.2.4 -- Previous Stable (2026-04)

Highlights: stability patch on top of v1.2.3 with 6 fixes from a Claude + Codex cross-review. EEPROM-compatible with v1.2.3.

Fixes: 2-second main-loop freeze on WiFi reconnect (`statusled.cpp`); detail-LED random loop without iteration cap (`detailleds.cpp`); `restoreFromSerial` now verifies the checksum (`handlers.cpp`); profile save no longer silently overwrites slot 0 when all 5 slots are full; serial-CLI confirmations no longer freeze the droid on USB disconnect (5 sites + central helper); servo limits validate `min < max`.

v1.2.5 -- Beta (2026-04)

Status: beta source for the v1.3.0 promotion. Not recommended for production -- use v1.2.4 for stable, v1.3.0 for the new feature set. Beta features (sequence recording, playlist, IR mapping) live in v1.3.0 with all bugs fixed.

v1.2.3 -- Stable (2026-03)

Status: previous-previous stable. Has the 6 carry-over bugs that v1.2.4 fixes -- use v1.2.4 instead. Reference for the v1 EEPROM layout (still readable by v1.3.0 migration path).

v1.1.0 -- Archive (2026-02)

Status: initial release, archived as historical reference. Basic 4-servo motion, simple eye animation, DFPlayer audio, rudimentary serial CLI. No WebUI, no IR learning, no sequence recording. Not recommended for any new build -- upgrade to v1.2.4 or v1.3.0.